Morph Channel

A Cell-Native Protocol Construction for Sponsored State Evidence and Local Channel Factories on CKB

Arthur Zhang

May 2026

Abstract

Problem Statement. Off-chain channel protocols must combine rapid off-chain state progression with enforceable unilateral settlement. On CKB, this problem has a distinctive form: Cell objects already carry explicit identity, data, lock scripts, type scripts, and outpoints. A channel architecture that merely imitates Bitcoin-style output reshaping would fail to use this state model directly. The research question of this paper is whether a channel can keep its funding identity stable while moving only signed state evidence during disputes.

Technical Contributions. This paper proposes Morph Channel, a CKB-native channel and channel-factory construction. It decomposes a channel into a stable funding anchor, moving state evidence, and a sponsor-owned publication layer. It defines a state-header-centric signing domain that excludes sponsor coin selection while binding chain context, channel identity, funding anchor, state number, asset registry, settlement descriptor, and challenge policy. It gives script-level validation sketches for the State Cell, vault lock, and bounded sponsor budget. It also introduces a partition-conservation algorithm under which occupied capacity, channel reserve, business CKB, and each xUDT type are checked separately, while publication fees are extracted only from sponsor-owned value. For bilateral channels, the first witness format is intentionally plaintext and auditable. For factories, authority moves to commitments, access manifests, local proofs, and touched leaves.

Formal Consequences. Under standard signature unforgeability and hash collision-resistance assumptions, Morph state signatures are non-replayable across channels and funding anchors. State Cell uniqueness is reduced to CKB's ordinary single-spend rule by making every non-initial state transition consume the currently live State Cell and create exactly one successor. If the relative challenge window covers detection, watchtower polling, transaction propagation, confirmation, fee bumping, and reorganisation margin, then a newer valid state can supersede an older published state before finalisation. Partition conservation implies zero channel-paid publication fees. A local factory exit is safe only under either full factory authorisation or a proof of non-interference for untouched participants.

Paper Type. This is a research-level protocol-design paper, not a CKB consensus-change proposal and not a claim of production completeness. The construction is deliberately limited to currently deployable CKB primitives: Cells, lock scripts, type scripts, witnesses, `cell_deps`, and `since`.

Keywords: CKB, Cell model, payment channels, channel factories, sponsored fees, partition conservation, xUDT, state evidence

Contents

1 Introduction and Motivation

Payment channels and state channels are motivated by a familiar tension: most state updates should remain off-chain, but unilateral settlement must remain enforceable on-chain when cooperation fails. The Lightning Network [1] gives the canonical payment-channel construction, while eltoo [2] isolates a replacement-based intuition: during a dispute, a newer mutually signed state should defeat stale evidence. Channel factories [3] extend this idea to shared funding structures that support many off-chain relationships.

CKB changes the substrate. Its ledger model is not merely a list of spendable outputs; it is a Cell model in which state objects have data, locks, type scripts, capacities, and stable outpoints [6, 7]. This suggests a different design question:

> What protocol structure is required so that CKB can express a channel whose funding object remains stable, while disputes move only signed state evidence?

This paper answers with Morph Channel. The construction keeps channel value anchored in stable Cells, moves a distinct State Cell during disputes, and pays publication fees from sponsor-owned funds rather than channel-owned value. The resulting architecture is not a direct port of Bitcoin eltoo. It borrows the newer-state-wins intuition, but realises it through CKB scripts, State Cells, relative `since`, and explicit value partitioning.

1.1 Scope

The paper focuses on script-level channel and factory architecture. It does not attempt to solve routing, liquidity discovery, gossip, privacy networks, watchtower markets, or factory governance. It also avoids any assumption of sub-second finality, DAG ordering, native channel transaction classes, or CKB node modification. All claims are bounded by existing CKB primitives.

The intended audit scope is construction audit. The question is not whether this draft is production-ready, but whether its protocol objects are sufficient, non-conflicting, and implementable with current CKB primitives. Security review should therefore proceed from threat model, to object authority, to transaction invariants, to deployability, and finally to empirical benchmarks.

1.2 Design Thesis

The central thesis is:

> *A CKB channel becomes simpler when funding identity, state authority, and publication fee responsibility are separate protocol objects.*

Funding identity answers which value object belongs to the channel. State authority answers how that value should be distributed. Sponsor authorisation answers who pays for the on-chain publication transaction. When these three objects are conflated, unilateral exit becomes dependent on wallet coin selection, fee bumping becomes a channel-state problem, and multi-asset conservation becomes difficult to audit.

2 Problem Statement and Current Snapshot

2.1 The Funding-State Coupling Problem

In many UTXO-oriented channel designs, settlement value and channel state are operationally coupled: a state update is represented by a new arrangement of future spend paths over the funding output. This coupling is natural in Bitcoin-like systems, but it is not inevitable on CKB. Since Cells can carry explicit typed state, a channel may instead maintain a stable funding anchor and move a separate state object.

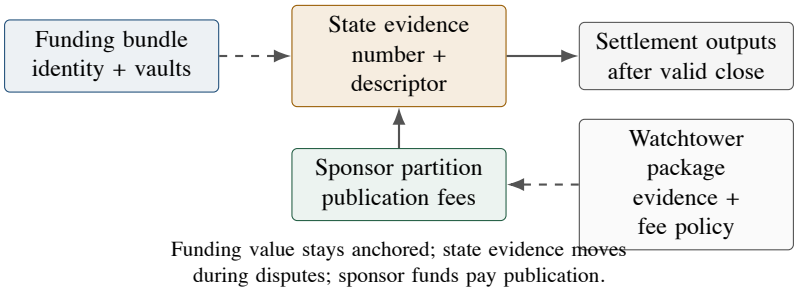

Figure 1: High-level decomposition. The channel value object, the moving dispute evidence, and the fee-paying sponsor funds are separate protocol objects.

The coupling problem is not merely aesthetic. If the object that holds value is repeatedly entangled with state-update mechanics, then fee payment, settlement authorisation, and asset conservation tend to collapse into one transaction template. Morph Channel separates them.

2.2 The Fee-Boundary Problem

A fee cap and a zero channel-paid fee policy are not equivalent. A fee cap says that channel value may be consumed up to a limit. A zero channel-paid fee invariant says that publication fees are not part of channel-owned value at all. CKB makes this distinction particularly important because capacity simultaneously represents storage resource, fee medium, and sometimes user-visible business value.

2.3 The Authority Object Problem

A channel protocol should not have multiple competing truth objects. In a poorly separated design, authority may drift among:

(1) the funding Cell;

(2) a plaintext witness;

(3) a local plaintext snapshot;

(4) a root commitment;

(5) a transaction body containing incidental sponsor inputs.

Morph Channel collapses authority into a signed state header plus the admissible witness or proof required by its mode. Transaction bodies remain reconstructible wrappers around that evidence.

3 Research Contributions and Design Principles

3.1 Main Contributions

This paper makes seven contributions.

C1. **Stable funding and moving evidence.** It defines a dual-track Cell construction in which channel value remains in stable funding and vault Cells, while disputes move a State Cell.

C2. **State-header-centric signing.** It specifies a canonical signing domain that binds identity and semantics while excluding sponsor coin-selection details.

C3. **Sponsored publication.** It defines a sponsor-owned publication partition and requires unilateral publication to be fee-bumpable without counterparty re-signing.

C4. **Partition conservation.** It replaces fee-cap reasoning with exact conservation of channel-owned value and fee extraction from sponsor-owned value.

C5. **Bilateral/factory witness separation.** It argues that bilateral channels should begin with plaintext witnesses, while factories require commitment-authoritative local proofs.

C6. **Challenge-window budgeting.** It derives the relative-`since` delay as a deployment parameter covering detection, polling, propagation, confirmation, fee volatility, and reorg margin.

C7. **Factory non-interference.** It identifies non-interference as the condition required before a factory action can safely reduce its signing set below all participants.

3.2 Design Principles

P1. **Script-level deployability.** All mechanisms must be expressible by deployable CKB scripts.

P2. **No hidden fee leakage.** Channel-owned capacity must not silently pay publication fees.

P3. **Narrow descriptors.** Settlement descriptors authorise output derivation and should not become an application runtime.

P4. **Measurability before defaulting.** Proof mode is not assumed cheaper until transaction size and CKB-VM cycles are measured.

P5. **Conservative factory coordination.** Locality is a verification property, not a substitute for multi-party authorisation.

P6. **Authentic state authority.** Vault finalisation is authorised by an authentic current State Cell under the expected scripts, not by bytes that merely decode as a state header.

P7. **Canonical challenge maturity.** V1 finalisation maturity is a canonical relative-block CKB `since` condition, not a raw integer threshold.

P8. **Value conservation for local proofs.** Non-interference localises a change; value-bearing right increases still require full consent or a real asset-delta witness.

P9. **Signed descriptor evolution.** A settlement descriptor instance may evolve only as part of a newly signed state; finalisation uses the descriptor commitment bound to the current authentic state.

P10. **Non-orphaning retirement.** Retiring state evidence must be coupled to value finalisation, or must leave verifiable evidence sufficient for value finalisation.

P11. **No unverifiable sponsor deadlines.** Sponsor-policy fields that require unavailable chain evidence must remain operator policy, or be inadmissible as script-enforced constraints.

4 System Model and Threat Assumptions

4.1 Ledger Model

Let $\mathcal{U}$ denote the set of live CKB Cells. A transaction consumes a finite input set $I \subseteq \mathcal{U}$, may reference read-only cells through `cell_deps`, and creates a finite output set O. CKB consensus verifies input availability, script validity, capacity constraints, and `since` maturity.

Assumption 1 (Base-layer correctness)**.** *CKB consensus correctly enforces transaction validity, script execution, occupied-capacity checks, and* `since` *semantics.*

Assumption 2 (Cryptographic soundness)**.** *The signature scheme is existentially unforgeable under chosen-message attacks, and the hash function used for state commitments is collision resistant.*

4.2 Actors

The protocol includes channel participants, optional watchtowers, ordinary wallets providing sponsor funds, and CKB miners/validators. A watchtower may observe chain events and publish newer state evidence, but it should not hold custody over channel-owned value.

4.3 Adversarial Goals

An adversary may attempt to:

A1. publish a stale state and let it finalise;

A2. replay a state signature across another channel or funding anchor;

A3. reference a descriptor-shaped but non-authoritative funding Cell;

A4. spend vault value without current state authorisation;

A5. make channel-owned capacity pay publication fees;

A6. confuse CKB reserve, CKB business balance, xUDT amounts, and sponsor change;

A7. exploit factory locality to harm untouched participants.

5 Formal Objects

5.1 Channel Configuration

A Morph channel configuration is a tuple

$$\Gamma = (\mathsf{chain}, \mathsf{cid}, \mathsf{participants}, \mathsf{fund}, \mathsf{assets}, \mathsf{descriptor}, \mathsf{challenge}, \mathsf{version}).$$

Here fund identifies the funding anchor, assets commits to the asset registry, descriptor commits to the current settlement-output derivation, and challenge specifies the relative finalisation policy. The descriptor commitment is state semantics: descriptor instances may advance with newly signed state, while descriptor grammar, version boundaries, and authorisation rules remain protocol semantics.

5.2 Stable Funding Bundle

Definition 1 (Funding Bundle)**.** *The funding bundle $\mathcal{F}$ consists of a channel identity Cell and zero or more vault Cells:*

$$\mathcal{F} = (\mathsf{Fund}, \mathsf{Vault}_1, \ldots, \mathsf{Vault}_k).$$

The bundle carries channel identity and channel-owned value. It is stable during ordinary off-chain state progression.

Definition 2 (Vault Authorisation)**.** *A vault Cell is admissible only if its lock or type script rejects every spend that is not authorised by the authentic currently valid State Cell and the committed settlement descriptor. A Cell whose data merely decodes as a state header is not state authority unless its script identity also matches the channel construction.*

Definition 3 (Current State Pointer)**.** *For a channel identifier* cid*, the current state pointer is the unique live State Cell that can be consumed by the next unilateral publication, supersession, or finalisation operation. The protocol does not require a global registry for this pointer; uniqueness follows from creating exactly one initial State Cell and requiring every state transition to consume the current one and create exactly one successor.*

Definition 4 (Initial State Uniqueness)**.** *A valid* FUND *transaction creates exactly one initial State Cell for the channel's funding anchor. The initial State Cell, the Fund Cell, and all vault locks must bind the same immutable channel genesis identity. A deployable instance should express this binding in State Cell type arguments, using a Type-ID-style uniqueness rule or an equivalent script-level construction available on current CKB.*

This uniqueness condition is a construction requirement, not a global lookup assumption. The scripts do not need to scan the chain for competing state tracks. Instead, they must make every valid vault spend and every valid State Cell transition refer to the same genesis identity, so a descriptor-shaped but non-authoritative State Cell cannot control the vaults.

5.3 Moving State Evidence

Definition 5 (State Evidence)**.** *State evidence is a signed object*

$$E_n = (\mathsf{Header}_n, \mathsf{Payload}_n, \Pi_n, \Sigma_n),$$

where Header_n *is the canonical state header,* $\mathsf{Payload}_n$ *is plaintext or committed state material,* Π_n *is an optional proof package, and* Σ_n *is the participant authorisation set.*

Definition 6 (Canonical State Header)**.** *The canonical state header contains:*

```
StateHeader {
  protocol_version
  chain_id
  signature_scheme_id
  channel_id_or_genesis_identity
  funding_anchor_identity
  state_number
  mode
  phase
  participants_commitment
  asset_registry_commitment
  settlement_descriptor_commitment
  descriptor_version
  payload_commitment
  challenge_policy_commitment
  state_layout_version
}
```

The signed message is

$$m_n = \mathsf{H}(\texttt{"CKB\_MORPH\_CHANNEL\_STATE\_V1"} \parallel \mathrm{canonical}(\mathsf{Header}_n)).$$

The version fields are part of the signed domain. The purpose is not only to prevent replay across channels, but also to prevent an old signature from being reinterpreted by a later descriptor, signature scheme, or state-layout parser.

5.4 Sponsor Partition

Sponsor funds are not part of the channel-owned value partition. They are ordinary wallet Cells, watchtower-owned Cells, or pre-authorised fee budgets. The script-enforced budget boundary should include channel identity, state-number range, maximum fee, expected state type, and clean change destination. Expiry windows, allowed sponsor sources, publishing cadence, and webhook policy may also be enforced by an operator or watchtower layer. If a deployment lacks script-verifiable evidence for such a field, the field must not be silently treated as a script-enforced safety condition.

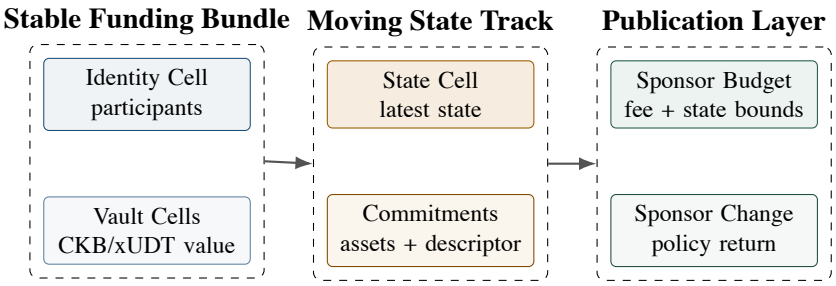

Figure 2: Protocol object map. The funding bundle holds channel identity and value; the state track carries authority; the publication layer pays the transaction fee.

6 Script-Level Validation Semantics

This section is intentionally pseudo-code rather than byte-level CKB-VM code. Its purpose is to state which checks must exist before the construction can be considered implementable.

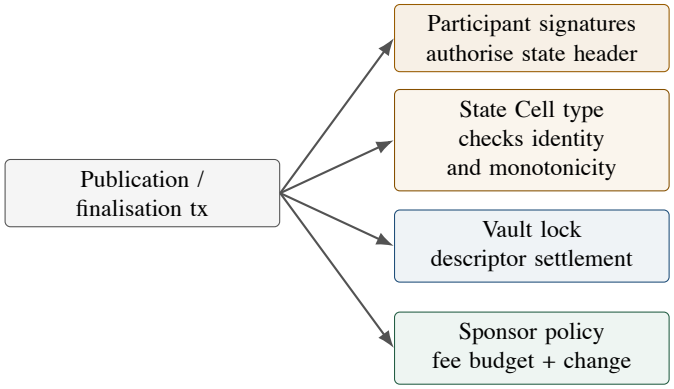

Figure 3: Authorisation boundaries. State signatures, vault spends, and sponsor fee authorisation are deliberately different checks.

6.1 State Cell Type Semantics

The State Cell type script owns state progression. It does not spend channel value; it verifies the identity and monotonicity of state evidence.

```
verify_state_transition(tx):
  old = exactly_one_input_state_cell(tx, channel_id)
  new = exactly_one_output_state_cell(tx, channel_id)
  old_header = parse_state_header(old.data)
  new_header = parse_state_header(new.data or witness)

  require new_header.protocol_version == old_header.protocol_version
  require new_header.chain_id == old_header.chain_id
  require new_header.signature_scheme_id
        == old_header.signature_scheme_id
  require new_header.channel_id == old_header.channel_id
  require new_header.funding_anchor_identity
```

```
      == old_header.funding_anchor_identity
 require new_header.asset_registry_commitment
      == old_header.asset_registry_commitment
 require descriptor_transition_admissible(
      old_header.settlement_descriptor_commitment,
      new_header.settlement_descriptor_commitment,
      old_header.descriptor_version,
      new_header.descriptor_version,
      witness)
 require new_header.challenge_policy_commitment
      == old_header.challenge_policy_commitment
 require new_header.state_number > old_header.state_number
 require new_header.phase == settling

 require referenced_funding_anchor(tx.cell_deps)
      matches new_header.funding_anchor_identity
 require signatures_valid(new_header, witness.signatures)
 require payload_or_proof_matches_commitment(new_header, witness)
 require partition_conservation(tx, new_header.asset_registry_commitment)
 require output_state_capacity_sufficient(new)
```

The descriptor transition predicate is deliberately narrow. It permits a new descriptor instance only when the new State Header is signed under the normal state-signing domain and the descriptor version and grammar remain within the protocol's accepted boundary. The vault later settles only against the descriptor commitment in the current authentic State Cell.

The "exactly one input, exactly one output" rule is the script-level expression of State Cell uniqueness. Mempool races are resolved by the ordinary CKB single-spend rule: competing transactions may exist, but at most one can consume the current State Cell on-chain.

6.2 Vault Lock Semantics

Vault locks own channel value. They must not accept a transaction merely because the transaction references the correct channel. They accept only settlement or materialisation authorised by the current state evidence.

```
verify_vault_spend(tx):
 op = classify_channel_operation(tx)
 require op in {FINALIZE, COOPERATIVE_CLOSE, SPLICE, MATERIALIZE}

 state = exactly_one_authentic_input_state_cell(
        tx, channel_id, expected_state_type, expected_state_lock)
 header = parse_state_header(state.data)
 require header.funding_anchor_identity matches this_vault.channel
 require signatures_or_phase_authorise(op, header, tx.witness)

 if op == FINALIZE:
  require header.phase == settling
  require state_input_relative_block_since_satisfies(header.challenge_policy)
  require this_vault matches header.payload_or_vault_commitment

 expected_outputs =
  derive_outputs(header.settlement_descriptor_commitment,
              header.payload_commitment,
              tx.witness)
 require canonical_outputs(tx.channel_outputs) == expected_outputs
 require partition_conservation(tx, header.asset_registry_commitment)
```

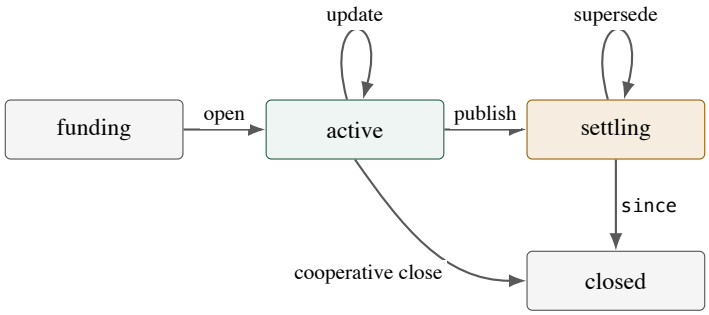

Figure 4: Morph Channel state machine. Publishing moves an active channel into the settling phase; newer signed evidence supersedes older settling evidence before finalisation.

This lock is the point at which "current valid state" becomes economically meaningful. A State Cell without vault enforcement is only evidence; it does not protect value.

6.3 Bounded Sponsor Budget Semantics

A sponsor budget may be implemented as a normal wallet signature, a watchtower-owned Cell, or a bounded policy Cell. The bounded policy form is the most delicate because it authorises another actor to spend capacity for publication.

```
verify_sponsor_budget_spend(tx):
  policy = parse_policy(this_budget_cell.data)
  header = parse_published_state_header(tx.witness)

  require header.channel_id == policy.channel_id
  require policy.min_state_number <= header.state_number
  require header.state_number <= policy.max_state_number
  require tx_fee(tx) <= policy.max_fee_per_tx
  require cumulative_fee_after(tx) <= policy.max_total_fee
  require sponsor_change_lock(tx) == policy.change_lock
  require no_channel_assets_in_sponsor_change(tx)
  require operation_is_publication_or_challenge(tx)
```

Expiry windows, allowed sponsor sources, scan cadence, and webhook policy are operator/watchtower policy in V1 unless the deployment gives the script verifiable evidence for those fields. A bounded sponsor script should reject, or otherwise make inadmissible, a finite script-level deadline for which it cannot verify the relevant chain evidence.

The policy is not an open wallet delegation. It is a bounded publication right scoped to one channel and one state interval.

7 Protocol State Machine

Let the channel phase be one of

$$\mathsf{Phase} = \{\mathsf{funding}, \mathsf{active}, \mathsf{settling}, \mathsf{closed}\}.$$

The lifecycle contains five operations:

$$\mathsf{FUND}, \quad \mathsf{OFFCHAIN\_UPDATE}, \quad \mathsf{PUBLISH}, \quad \mathsf{SUPERSEDE}, \quad \mathsf{FINALIZE}.$$

8 Transaction Semantics

8.1 FUND

FUND creates the stable funding bundle and the initial State Cell. The funding transaction may be paid by ordinary wallet inputs. After funding, channel-owned value enters the channel partition and should not be used for publication fees.

The initial State Cell is the only on-chain active pointer created by the channel. Its type arguments or equivalent identity mechanism must bind the same funding-anchor identity used by the Fund Cell and vault locks. A transaction that attempts to create two initial State Cells for the same channel genesis identity is invalid under the funding script profile.

8.2 OFFCHAIN_UPDATE

An off-chain update produces a new state evidence object with a strictly larger state number. It does not consume CKB Cells. Participants exchange signatures over the canonical state header. State numbers are strictly monotonic, but not necessarily consecutive: a valid supersession may move from n to $n + k$ if the newer state is properly authorised and the sponsor policy admits that state interval.

Off-chain active updates do not create on-chain active State Cells. Once a unilateral publication reaches the chain, the channel is in the settling phase. Further on-chain progress is either supersession by a newer settling State Cell or finalisation after the relative challenge delay.

8.3 PUBLISH and SUPERSEDE

The same transaction pattern handles initial unilateral publication and challenge:

```
PUBLISH_STATE:
  inputs:
    StateCell(n, active_or_settling)
    SponsorCell*
  cell_deps:
    FundingAnchor
    VaultCell*
  outputs:
    StateCell(n', settling)
    SponsorChange*
  validity:
    n' > n
    signatures over canonical Header(n') are valid
    funding identity matches referenced anchor
    channel partition is conserved
```

The transaction body is not the protocol authority. It is a reconstructible carrier for state evidence plus sponsor fee authorisation. Under competing tx-pool spends, a challenger may rebuild the transaction against the currently live State Cell.

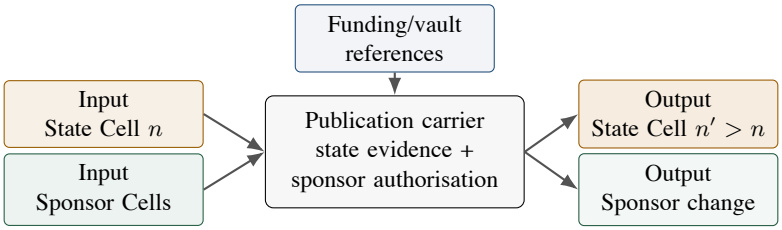

Figure 5: Publication and supersession transaction shape. The channel evidence can be reused while the sponsor inputs and fee choice are rebuilt.

8.4 FINALIZE

FINALIZE consumes the current settling State Cell and the relevant vault Cells after the relative `since` delay has matured. It creates the settlement outputs authorised by the committed settlement descriptor. Retiring the state track without finalising or otherwise leaving verifiable finalisation evidence is invalid, because it would orphan channel value. Cooperative close is a special path that carries explicit authorisation from the required participants and does not require a challenge delay.

9 Partition Conservation

For every post-funding transaction, partition the transaction's values into channel-owned value C and sponsor-owned value S. Read-only `cell_deps` do not participate in value accounting. The high-level rule is:

$$C_{\text{in}} = C_{\text{out}}, \qquad S_{\text{in}} - S_{\text{out}} = \text{fee}, \qquad C \cap S = \emptyset.$$

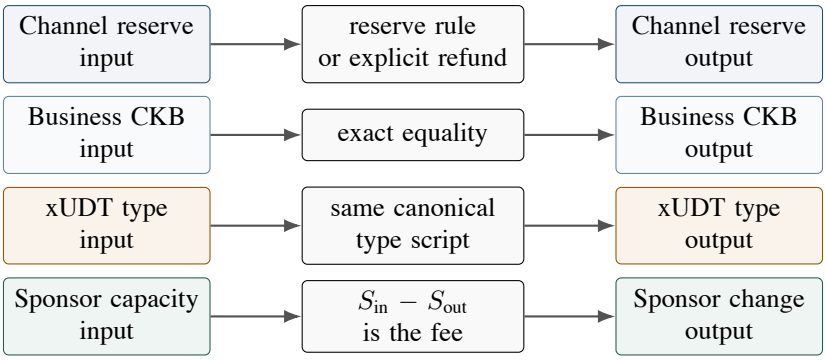

Figure 6: Partition conservation. Capacity needed for storage, user-visible CKB, each xUDT type, and sponsor fees are checked as separate lanes.

Invariant 1 (Partition Conservation)**.** *Channel-owned value is exactly conserved across post-funding publication and challenge transactions. Any fee deficit is borne exclusively by the sponsor partition.*

9.1 Capacity-Aware Conservation Algorithm

CKB capacity requires a finer rule than a single scalar balance. A channel-owned CKB Cell has two logically distinct components:

$$\mathsf{capacity}(c) = \mathsf{reserve}(c) + \mathsf{business\_ckb}(c), \qquad \mathsf{reserve}(c) \geq \mathsf{occupied}(c).$$

The reserve component exists so the Cell can exist. The business component is user-visible CKB value. A valid transition must not silently convert reserve into fee, sponsor change, or business balance unless the descriptor explicitly authorises a reserve refund.

For xUDT accounting, an asset type is the canonical identity of the full type script, for example the hash of its canonical encoding. It is not a symbol, name, display metadata, or wallet-level asset label.

```
partition_conservation(tx, asset_registry):
  classify every input/output cell as:
    SPONSOR
    CHANNEL_RESERVE
    BUSINESS_CKB
    BUSINESS_XUDT(asset_type)
    STATE_CARRIER
    UNRELATED
```

Partition	Source	Role	Fee payer
Sponsor capacity	ordinary wallet or budget Cells	transaction fees and change	yes
Channel reserve	channel-owned CKB capacity	Cell occupancy and reserve refund	no
Business assets	CKB or xUDT balances	participant settlement value	no

Table 1: Value partitions in Morph Channel.

```
require UNRELATED cells are not read by channel scripts
      and do not contribute to channel conservation
require all channel output cells satisfy
      capacity(output) >= occupied_capacity(output)

reserve_in = sum(channel_reserve_amount(input))
reserve_out = sum(channel_reserve_amount(output))
require reserve_out + authorised_reserve_refund == reserve_in

business_ckb_in = sum(business_ckb_amount(input))
business_ckb_out = sum(business_ckb_amount(output))
require business_ckb_out == business_ckb_in

for each asset_type in asset_registry.xudt_types:
  type_script = asset_registry.type_script(asset_type)
  require asset_type == hash(canonical(type_script))
  require sum_xudt(input, type_script)
        == sum_xudt(output, type_script)
  require every xUDT output for asset_type
        carries type_script

require sponsor inputs are not classified as channel reserve
sponsor_in = sum_capacity(sponsor_inputs)
sponsor_out = sum_capacity(sponsor_outputs)
require sponsor_in - sponsor_out == tx_fee(tx)
require sponsor_outputs carry no channel business assets
require sponsor_outputs carry no registered xUDT type
require sponsor_outputs do not use channel vault locks
      unless explicitly classified by the descriptor
```

The algorithm separates four questions that are otherwise easy to confuse: whether a Cell can exist, whether channel reserve is conserved or refunded, whether CKB business value is conserved, and whether each xUDT type is conserved under its own type script. Unrelated Cells may appear only when they are irrelevant to channel script reads and irrelevant to conservation. They must not be used as hidden witnesses for channel authority.

Proposition 1 (Zero Channel-Paid Publication Fees)**.** *If partition conservation holds for a post-funding transaction, then publication fees cannot reduce channel-owned value.*

Proof. CKB transaction fees are represented by the difference between input capacity and output capacity. Since channel reserve, business CKB, and each registered xUDT type are conserved within the channel partition, no fee deficit can be charged to channel-owned value. The only permitted capacity deficit is $S_{\text{in}} - S_{\text{out}}$, the sponsor partition. □

10 Settlement Descriptor

The settlement descriptor is a protocol object, not a general-purpose application language. It commits to the admissible output derivation rule for settlement and local materialisation. A minimal descriptor

specifies:

(1) participant destination policies;

(2) per-asset allocation rules;

(3) occupied-capacity requirements for outputs;

(4) reserve refund rules;

(5) required xUDT type scripts;

(6) sponsor change classification;

(7) descriptor version and upgrade boundary.

Invariant 2 (Descriptor Boundedness)**.** *The descriptor must authorise settlement, splice, factory materialisation, and reserve refund. It must not introduce unbounded application execution into channel witnesses.*

11 Bilateral Witness Mode

For a two-party channel, the first witness format should remain plaintext. This is not a claim that privacy is irrelevant; it is a complexity boundary. Bilateral state is typically small: balances, conditional payments, timeout information, and shutdown policies. Plaintext witnesses make force-close transactions auditable and make early prototypes easier to test.

Plaintext is not authority by itself. The authoritative object remains the signed state header and its payload commitment. A validator checks that the plaintext payload deterministically induces the committed payload hash and satisfies the descriptor and conservation rules.

12 Factory Proof Mode

Factories require a different authority model. A factory may contain many participants, balances, subchannels, reserves, and local exit paths. Full plaintext disclosure on every local exit is neither private nor obviously scalable. Factory authority should therefore be commitment-first.

Definition 7 (Factory State)**.** *A factory state is a state header plus a set of roots:*

```
FactoryState {
  state_header
  balances_root
  subchannels_root
  membership_root
  reserve_root
}
```

Definition 8 (Factory Witness)**.** *A factory witness contains an access manifest, a proof bundle, touched leaf payloads, and the required signatures:*

```
FactoryWitness {
  state_header
  access_manifest
  proof_bundle
  touched_leaf_payloads
  signatures
}
```

Definition 9 (Transition Footprint)**.** *The transition footprint of a factory action is the set of leaves read, written, inserted, removed, or proven absent by that action.*

Definition 10 (Factory Right)**.** *A factory right is any participant-relevant claim that may be changed by a local factory action:*

$$\mathsf{FactoryRight} = \{\mathsf{balance}, \mathsf{reserve\_claim}, \mathsf{membership}, \mathsf{exit\_path}, \mathsf{sponsor\_budget\_claim}\}.$$

Definition 11 (Non-Interference)**.** *A factory action is non-interfering with respect to untouched participants if the proof bundle establishes that all their factory rights are unchanged, including rights that depend on shared reserve, membership, exit path, or public sponsor-budget state.*

The difficult object in a factory is therefore not the Merkle proof alone, but the rights-dependency graph. A touched balance leaf may also affect reserve liquidity, local exit eligibility, or sponsor-budget allocation. A proof of non-interference must either cover those dependencies or the action must fall back to full-participant authorisation.

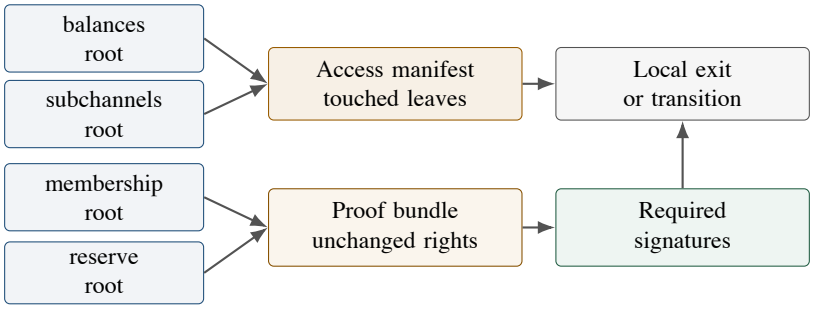

Figure 7: Factory proof mode. A local transition must expose the leaves it touches and prove that the relevant untouched rights are unchanged.

Proposition 2 (Conservative Factory Authorisation)**.** *If a factory action cannot prove non-interference, then a conservative first implementation should require signatures from all factory participants. If non-interference is proven, the signing set may be reduced to the affected participants only when the touched rights also satisfy participant authorisation, local validity, and value conservation.*

Proof. Without non-interference, untouched leaves may still depend on shared membership, reserve, or exit-right state. A local signature set would not authorise those possible effects. Full-participant signatures authorise global effects. A non-interference proof removes only the effects on untouched rights; it does not by itself authorise minting a balance, increasing a reserve claim, deleting membership, or releasing value from a vault. □

13 Challenge-Window Semantics

The challenge window is not a paper constant. It is a deployment parameter derived from CKB confirmation behaviour and watchtower liveness.

Let Δ denote the finalisation delay encoded by relative `since`. A conservative policy should satisfy:

$$\Delta \geq \Delta_{\text{detect}} + \Delta_{\text{poll}} + \Delta_{\text{build}} + \Delta_{\text{confirm}} + \Delta_{\text{margin}}.$$

The terms respectively represent detection confirmations, watchtower polling interval, transaction construction and propagation, confirmation of the newer-state transaction, and a margin for fee bumping and reorganisation risk. V1 uses a canonical relative block-number `since`. A future deployment may choose epoch- or timestamp-based maturity only if the mode, metric, reserved bits, and value are canonicalised and tested against current CKB hardfork semantics [8, 9, 10]; raw integer comparison is not a valid challenge-window check.

Parameter	Deployment evidence	Audit question
Detection depth	chosen confirmation depth, for example 1, 3, or 6 confirmations	when does a watchtower treat stale evidence as actionable?
Polling interval	watchtower service objective	is there enough time between observation and finalisation?
Build and broadcast time	wallet, RPC, and network measurements	can a replacement carrier be assembled against the live State Cell?
Confirmation target	measured CKB confirmation behaviour	can the newer state confirm before the deadline?
Fee-bump margin	sponsor budget multiple	is there budget for conservative publication plus emergency rebuild?
Reorganisation margin	conservative deployment choice	does the delay tolerate expected reorg risk?

Table 2: Deployment profile required for selecting a challenge window.

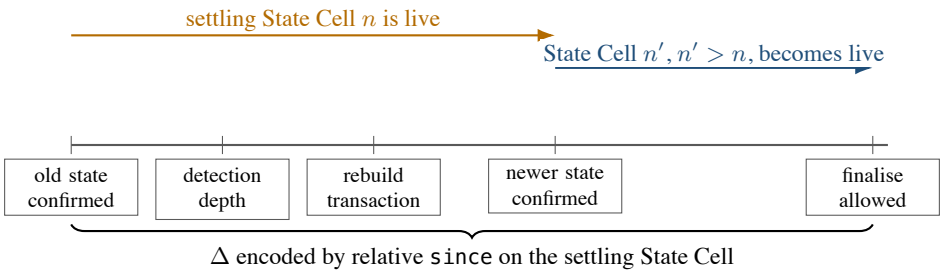

Figure 8: Challenge-window timeline. Safety depends on confirmed supersession before the relative finalisation delay matures, not on tx-pool visibility alone.

13.1 CKB `since` and Mempool Reality

In CKB, relative `since` is checked on transaction inputs. For Morph finalisation, the relevant input is the currently live settling State Cell. Therefore a finalisation transaction should set `since` on the State Cell input and the State type or vault lock should check that the value is an encoded relative-block condition matching the committed challenge policy. A naked numeric value is not equivalent to a relative delay.

Safety should be defined by confirmation before the deadline, not by mempool presence. A challenger may observe an old state in the tx-pool, but the protocol should not require responding until an old settling State Cell is actually confirmed or reaches the configured detection depth. If the old state confirms, the challenger rebuilds a newer-state transaction against the now-live settling State Cell. If a newer-state transaction remains unconfirmed, the watchtower must be able to rebuild it with different sponsor inputs or a higher fee, because the channel signature does not commit to sponsor coin selection.

This does not assume Bitcoin-style replace-by-fee semantics. Fee bumping here means reconstructing a fresh transaction body against the current live State Cell and current sponsor funds. Once a competing transaction consumes that State Cell, all stale transaction bodies become invalid and must be rebuilt.

```
watchtower_response_loop:
  observe chain tip and tx-pool
  if confirmed settling StateCell(n) reaches detection_depth:
    latest = local_state_package(channel_id)
    if latest.state_number > n:
      tx = build_publish_state_tx(
            input_state = live StateCell(n),
            state_package = latest,
            sponsor_policy = available_budget(),
            fee_rate = conservative_fee_rate())
```

```
broadcast(tx)
while tx not confirmed and finalisation_deadline not near:
  if fee market or sponsor set changes:
    tx = rebuild_against_live_state_cell()
    broadcast(tx)
```

The watchtower policy must reserve enough fee budget for at least one conservative publication and one emergency rebuild. If the deployment expects volatile fees, the budget should support additional rebuild attempts. This is an operational requirement, not merely a wallet convenience.

Proposition 3 (Newer-State Supersession)**.** *Suppose a settling State Cell at number n is live. If an honest party publishes and confirms valid evidence $E_{n'}$ with $n' > n$ before finalisation of E_n, then E_n cannot be the final settlement authority.*

Proof. The supersession transaction consumes the live settling State Cell at n and creates a new settling State Cell at n'. Once consumed, the n State Cell cannot be used as an input to finalisation. The finalisation rule applies only to the currently live settling State Cell after its own relative `since` delay matures. □

14 Signature Domains and Authorisation Boundaries

Morph separates three authorisation domains:

(1) state authorisation: participant signatures over the canonical State Header;

(2) value authorisation: vault locks accepting only descriptor-authorised settlement or materialisation;

(3) fee authorisation: sponsor signatures or bounded sponsor policies.

Lemma 1 (Domain-Separated Anti-Replay)**.** *If protocol version, signature-scheme identity, chain identity, channel identity, funding anchor identity, asset registry, descriptor commitment, descriptor version, challenge policy, and state number are included in the signed State Header, then a valid state signature cannot be replayed as a valid state for another channel context or later protocol interpretation.*

Proof. Any change to the protocol version, signature scheme, chain, channel, funding anchor, asset registry, descriptor, descriptor version, challenge policy, or state number changes the canonical State Header and hence the signed message. Under signature unforgeability, the adversary cannot produce a valid participant signature for the modified context. □

15 Watchtower and Recovery Objects

A watchtower is a liveness delegate, not a custodian. It stores the latest state package and publishes newer evidence when stale evidence appears on-chain. A state package is richer than a bare root but leaner than a permanent plaintext dump:

$$\mathsf{StatePackage} = (\mathsf{Header}, \mathsf{Payload}\text{ or roots}, \Pi, \Sigma, \mathsf{sponsor\_policy}).$$

The script-enforced sponsor boundary must be bounded by channel identity, state-number range, maximum fee, expected State Type, and change destination. Expiry windows, allowed sponsor sources, and publishing cadence are operator/watchtower policy in V1 unless a deployment provides script-verifiable evidence for those fields; finite script-level deadlines without such evidence should be rejected rather than treated as consensus-enforced policy. The watchtower should not be able to redirect channel-owned value or drain arbitrary wallet funds.

16 Safety Properties

Proposition 4 (Funding Anchor Uniqueness)**.** *If the State Header binds a funding anchor identity and scripts verify that referenced* `cell_deps` *match that identity, then a descriptor-shaped but non-authoritative Cell cannot serve as the channel's funding anchor.*

Proof. The script compares the referenced Cell identity with the identity committed in the State Header. A different Cell, even with similar data, has a different outpoint or genesis identity and is rejected. □

Proposition 5 (Vault Spend Safety)**.** *If each vault lock accepts only settlement or materialisation transactions authorised by an authentic current State Cell and descriptor-admissible outputs, then channel-owned value cannot be spent through ordinary wallet paths or fake state-header bytes.*

Proof. Every spend of channel-owned value must satisfy the vault lock. By assumption, the lock rejects transactions lacking authentic current state evidence and descriptor-admissible outputs. Ordinary wallet signatures alone are insufficient, and a non-authoritative Cell that only imitates the State Header layout does not satisfy the state-evidence predicate. □

Proposition 6 (Recoverable Unilateral Settlement)**.** *If a participant or watchtower stores a valid State Package, has access to sponsor funds within policy, and the challenge delay satisfies the response-budget inequality, then it can reconstruct a publication transaction without fresh counterparty cooperation.*

Proof. The channel signature excludes sponsor inputs and fee rate, so the publication transaction may be rebuilt using available sponsor funds. The State Package supplies the signed state evidence and required witness or proof. The response-budget inequality provides time for detection, construction, propagation, and confirmation. □

17 Evaluation Agenda

This draft does not claim empirical closure. It defines a benchmark agenda:

E1. bilateral plaintext witness size under realistic payment states;

E2. factory proof size under realistic touched-footprint distributions;

E3. CKB-VM cycles for signature verification, hash verification, proof verification, and xUDT type-script composition;

E4. challenge reliability under confirmation-depth, fee-volatility, and watchtower-polling parameters;

E5. sponsor-policy failure cases, including insufficient budget, inadmissible finite script-level expiry, operator-level expiry, and wrong change destination;

E6. State Cell uniqueness tests under concurrent publication and supersession attempts;

E7. vault-lock rejection tests for spends lacking current state evidence;

E8. reserve refund, occupied-capacity, business-CKB, and per-xUDT conservation edge cases;

E9. executable negative tests corresponding to the audit matrix in Appendix A;

E10. comparison between full plaintext factory disclosure and local proof verification.

18 Related Work and Positioning

Lightning [1] represents the penalty-based payment-channel family. Eltoo [2] provides the update-by-replacement intuition, but depends on Bitcoin sighash changes. Channel factories [3] motivate shared funding and local exit questions. State-channel frameworks such as Perun [4] show the broader adjudication pattern in which off-chain evidence is made enforceable by an on-chain dispute procedure. CKB community work on generic payment channels [5] is a nearer point of comparison, because it studies channel composability directly on the Cell model.

CKB's Cell model [6] and transaction structure [7] provide explicit state objects and read-only dependency references. CKB's `since` field [8, 9, 10] provides relative lock semantics needed for challenge windows. SUDT [11] and xUDT [12] motivate per-asset conservation rather than a single undifferentiated balance.

Morph's distinctive claim is not that CKB can copy Lightning, nor that script-level channels replace existing work. The claim is narrower: CKB can host a channel object whose funding identity, state evidence, fee publication, and factory-local proof objects are separated at the script level without changing base-layer consensus.

19 Limitations and Open Questions

The construction remains incomplete in several respects:

O1. Factory coordination remains an off-chain protocol problem.

O2. Proof mode may not be cheaper than plaintext until measured.

O3. Mainnet challenge delays require empirical confirmation and fee data.

O4. Descriptor versioning must balance upgrade flexibility against recoverability.

O5. Different xUDT implementations may impose different constraints.

O6. The paper now specifies script-level semantics, but not byte-level Molecule schemas or complete CKB-VM implementations.

O7. Factory non-interference still requires a formal rights-dependency schema before reduced signing sets are safe.

O8. Typed reduced exits are admissible only when asset type, amount, child vault, settlement descriptor, and factory-vault change are completely bound; broader typed-asset compatibility remains future work.

20 Conclusion

Morph Channel proposes a CKB-native channel architecture built around stable funding anchors, moving signed state evidence, sponsored publication, partition conservation, and local factory proofs. The design is intentionally conservative: it does not require CKB node modifications, does not assume fast finality, does not treat Merkle proofs as automatically cheaper, and does not claim that factory coordination is solved by an on-chain format.

The value of the proposal is that it turns several ambiguous channel concerns into explicit protocol objects. Funding identity, state authority, fee responsibility, settlement descriptors, and factory transition footprints become independently auditable. This is the necessary first step before implementation benchmarks can decide whether Morph Channel should remain a research direction or mature into a production CKB channel protocol.

A Audit Matrix

This appendix states the construction audit target in testable form. Each row should become at least one positive test and one negative test in any prototype implementation.

ID	Invariant	Enforced by	Positive case	Negative case
S1	One live State Cell controls the channel pointer	State Cell type plus CKB single-spend	supersede $n \to n+1$ by consuming current State Cell	transaction creates two successor State Cells
S2	Funding anchor identity is canonical	State Header plus `cell_deps` check	referenced anchor matches committed genesis identity	descriptor-shaped fake anchor is referenced
S3	State numbers are strictly monotonic	State Cell type script	supersede $n \to n+k$ with valid signatures	publish lower or equal state number
S4	State retirement does not orphan value	State and vault authority coupling	retirement finalises value or leaves verifiable finalisation evidence	State Cell is consumed while matching vault value remains live without evidence
V1	Vault value follows current state evidence	Vault lock and settlement descriptor	finalise after valid `since` delay	wallet-only spend of vault value
V2	Vault authority requires an authentic State Cell	State/Vault script identity checks	State Cell under expected scripts controls finalise	ordinary Cell carries decodable State Header bytes
V3	Descriptor evolution is signed state semantics	State signature and descriptor commitment	newer signed state commits to an admissible descriptor instance	finalise supplies an unsigned replacement descriptor
P1	Channel-owned capacity never pays publication fees	partition conservation	sponsor capacity pays fee and receives change	reserve reduced by transaction fee
P2	Reserve and business CKB are not confused	reserve and business-CKB ledgers	authorised reserve refund is explicit	reserve becomes hidden business balance or fee
X1	xUDT value is conserved by canonical type script	asset registry and xUDT type check	same canonical type script and amount	same symbol or metadata with different type script
G1	Sponsor change is uncontaminated	sponsor policy and partition classifier	sponsor change contains only sponsor capacity	sponsor change carries registered xUDT or vault lock
G2	Unverifiable sponsor deadlines are not script policy	sponsor policy boundary	unbounded script policy or verifiable deadline evidence	finite deadline is accepted without chain-verifiable evidence
U1	Unrelated Cells cannot influence channel validity	script read set and conservation classifier	unrelated wallet input pays no channel role	helper Cell is read by channel scripts without classification
C1	Challenge safety is confirmation-based	deployment profile and `since` policy	newer state confirms before deadline	stale state finalises while newer tx is only in mempool
C2	Challenge maturity is canonical relative block `since`	State/Vault since parser	encoded relative block delay matures	raw absolute integer is accepted as a delay
F1	Factory locality preserves untouched rights	proof bundle and rights-dependency graph	touched leaves prove affected rights only	shared reserve or exit right changes without authorisation
F2	Factory locality is not mint authority	local validity and conservation predicates	value right decreases or vault-delta-bound increases	Merkle-only proof increases ReserveClaim or Balance

Table 3: Construction audit matrix for Morph Channel.

References

[1] Joseph Poon and Thaddeus Dryja. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. https://lightning.network/lightning-network-paper.pdf.

[2] Christian Decker, Rusty Russell, and Olaoluwa Osuntokun. eltoo: A Simple Layer2 Protocol for Bitcoin. https://blockstream.com/eltoo.pdf.

[3] Conrad Burchert, Christian Decker, and Roger Wattenhofer. Scalable Funding of Bitcoin Micropayment Channel Networks. https://tik-old.ee.ethz.ch/file/
/49218d6b9325ddb4f18aef8830ec567b/1/scalable_funding.pdf.

[4] Stefan Dziembowski, Lisa Eckey, Sebastian Faust, and Daniel Malinowski. Perun: Virtual Payment Hubs over Cryptocurrencies. Cryptology ePrint Archive, Report 2017/635. https://eprint.iacr.org/2017/635.

[5] Nervos Talk. A Generic Payment Channel Construction and Its Composability. https://talk.nervos.org/t/a-generic-payment-channel-construction-and-its-composability/4697.

[6] Nervos RFC 0002. CKB Cell Model. https://nervosnetwork.github.io/rfcs/rfcs/0002-ckb/0002-ckb.html.

[7] Nervos RFC 0022. CKB Transaction Structure. https://nervosnetwork.github.io/rfcs/rfcs/0022-transaction-structure/0022-transaction-structure.html.

[8] Nervos RFC 0017. Transaction valid since. https://nervosnetwork.github.io/rfcs/rfcs/0017-tx-valid-since/0017-tx-valid-since.html.

[9] Nervos RFC 0028. Change since relative timestamp. https://github.com/nervosnetwork/rfcs/blob/master/rfcs/0028-change-since-relative-timestamp/0028-change-since-relative-timestamp.md.

[10] Nervos RFC 0030. Ensure index is less than length in since. https://github.com/nervosnetwork/rfcs/blob/master/rfcs/0030-ensure-index-less-than-length-in-since/0030-ensure-index-less-than-length-in-since.md.

[11] Nervos RFC 0025. Simple UDT. https://nervosnetwork.github.io/rfcs/rfcs/0025-simple-udt/0025-simple-udt.html.

[12] Nervos Talk. RFC: Extensible UDT. https://talk.nervos.org/t/rfc-extensible-udt/5337.